Knowing What You Need Is Not Enough: The Policy Bottleneck in Homeostatic Agents

Anonymous Author(s)
Anonymous Institution
anonymous@email.com

Abstract

A homeostatic agent acts to keep internal drives near their setpoints, so its behaviour hinges on the satiation signal g: how much each outcome quiets each drive. Hand-designing g risks an agent that regulates flawlessly toward the wrong thing. We ask whether g can instead be learned from observable consequences, in a domain anyone can picture — a student with forty study sessions before an exam, choosing how hard to push. Sweeping environmental adversity with hashed predictions and paired analysis over 100 profiles, we decompose the gap between the homeostatic agent and a simple state estimator. Under the agent's hand-chosen policy and at ϕ=0, signal quality is the smaller term: an expected-gain oracle buys 7.7% of the gap against 25.0% for re-tuning six policy coefficients, with over half surviving both. But the two are complementary rather than competing — the oracle's advantage is larger under a harder policy search ($+0.121$ vs. $+0.047$), though a direct interaction test over five searches per budget does not establish the growth (p=.11) — so what plausibly binds is the channel that turns satiation into action. Regulation earns a narrower role than we first claimed: viability is secured by the rest mechanism, whose trigger the drives do help compute. Diagnosing why sent us back to our own applicability framework, which we find under-specified, and we propose a sixth condition.

1 Introduction

What moves an agent to act? In biology, action is not a default state — it emerges when internal variables leave viable ranges [1]. Yet dominant paradigms in agent design optimize external objectives and rarely treat internal regulatory integrity as first-class: what the agent itself *needs* is absent. That absence shows in where engineering effort has gone: from prompt engineering to context engineering to harness and loop engineering, successive scaffolding layers around a capable model. Each adds capability; none adds motivation. This line bets on a different move, from **agent engineering** to **agent cybernetics**: less effort adding capabilities, more inserting the structure that decides when and why a capability should be exercised at all.

Prior work in this line established feasibility. The Pro-Action Operator Γ showed that coupled homeostatic drives with LLM reasoning produce behaviour differentiated by opponent type in iterated social dilemmas (ICML 2026, anonymized). A single-drive variant produced emergent regulatory behaviour in debate facilitation without reinforcement learning — and surfaced a structural limitation: g was hand-designed, and when misaligned the system regulated toward the wrong attractor (SANLP 2026, anonymized). Tied to a structural metric it regulated participation but not semantic fidelity; tied to an LLM judge, context collapse *quintupled*. The loop worked — toward the wrong thing.

We call this the **Satiation Signal Problem (SSP)**: how is g defined so the system regulates toward a desired external goal? It is one problem inside the broader agenda of agent cybernetics, not the agenda itself — but it gates the rest, because the mechanism is value-neutral and the signal feeding it

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

decides what it regulates toward. So we ask: **can a homeostatic agent learn g from the observable consequences of its own actions, and use it to regulate toward an externally useful goal?** The domain is a student with forty study sessions before an exam, choosing difficulty guided by two drives — one pushing, one braking — while never observing its own ability, fatigue or frustration.

A single operating point cannot answer this. An earlier iteration produced a null admitting two readings: the signal does not matter, or the environment never applied pressure enough to make it matter. We therefore sweep the pressure, hash predictions before execution, gate every sweep point, and analyze 100 profiles as the paired design it is. What comes out is a decomposition rather than a verdict. Holding the hand-chosen policy fixed, an expected-gain oracle buys 7.7% of the gap to a state estimator while re-tuning six coefficients buys 25.0% — but the two are not independent, and that is the finding. The oracle's advantage is larger where the policy has been searched hard ($+0.121$, p=.022) than at a smaller budget ($+0.047$, p=.050), though a direct test of the difference over five searches per budget does not establish it ($+0.074$, CI $[-0.026, +0.175]$, p=.11): a satiation signal appears worth little until the channel can carry it.

Where regulation earns a role is viability, and there too we narrow an earlier claim: with rest disabled the homeostatic agent still reaches 93.5% dropout against the estimator's 100%.

Four contributions follow. **(1)** The decomposition, with paired effect sizes and confidence intervals, replicated on the goal metric. **(2)** The matched-tuning result: what signal quality is worth depends on how hard the policy is searched. **(3)** A methodology for architectures making claims about *regimes* — dose–response sweeps with per-point gating, hashed predictions, multiplicity-corrected existence tests, and multi-seed tuning with held-out validation. **(4)** A revision of our own applicability framework, including a sixth condition. We also report, in the body rather than a footnote, eight of our own claims that stricter protocols forced us to withdraw.

2 Background and Related Work

In game AI, Zubek [8] synthesized the needs-based approach exemplified by The Sims — drawing on robotics and industry practice rather than that title's implementation — in which agents act to satisfy a vector of decaying needs. It produces believable behaviour without scripting, but the mappings are hand-authored: the agent never discovers what satisfies it. What if it had to learn them?

That question leads to biology. Cannon [1] formalized homeostasis as maintaining internal variables within viable ranges through negative feedback — robust enough to underpin temperature, glucose and fluid regulation across all vertebrates, but correcting *after* deviation is detected. Sterling and Eyer [2] extended this with allostasis: predictive adjustment based on anticipated demand. The distinction is temporal, and it predicts a failure mode our agent encounters.

Keramati and Gutkin [3] bridged these with RL, defining primary reward as the reduction in homeostatic deviation and proving that maximizing discounted reward is equivalent to minimizing discounted deviation — reward-seeking *is* stability-seeking. Yet the drive function is fixed: the agent learns *which actions* reduce deviation, not *how much each outcome satisfies each drive*. That mapping is prescribed, and our work picks up exactly there. Ashby's Law of Requisite Variety [4] tells us where to look when it fails: a regulator needs as much variety as the disturbances it must counter, and our central negative result turns out to be a variety failure in the channel rather than in the drives. Further context, including the successor-representation connection and a critique of setpoints as relocated norms, is in Appendix A.

3 When Is a Problem Regulatory?

A framework that cannot say where it does *not* apply is not a framework. We hold that a problem is regulatory when it exhibits five properties: **(1)** variables that must be sustained within viable ranges rather than maximized — sleep, where neither zero nor sixteen hours beats seven; **(2)** antagonistic tensions that cannot be resolved once and forgotten, as a training programme balances stimulus against recovery; **(3)** non-linear dynamics, where crossing a threshold does not extrapolate smoothly — overtraining produces an injury, not a gradual decline; **(4)** situated information the agent observes about itself that an external orchestrator cannot, since a coach sees your lifts but only you feel the joint about to give; and **(5)** an objective learning signal.

Our experiment *manipulates* (1) and (3) through the adversity sweep. Conditions (2) and (4) are only *diagnosed* post hoc, and we are careful in Section 6 not to treat this experiment as evidence about mechanisms it never varied. Section 6 also revises the list, including a sixth condition we did not anticipate. The everyday version of all this: push to your maximum every day and you quit; never push and you stagnate. The right behaviour is not a schedule but a dynamic balance, adjusted from how your body responds.

The target band. We measure the difficulty band that maximizes expected learning gain, which is inspired by but narrower than Vygotsky's Zone of Proximal Development [9]. Our environment models no scaffolding, so we call it the **optimal-challenge band**, with time-in-band (TIB) as its metric, and reserve the ZPD label for the motivating construct.

4 Experimental Design

4.1 The agent: two drives, and what they feel like

The agent is given *needs*, not goals — two of them, both deficits that grow until something quiets them. δ_{prog} is the itch of coasting, what you feel in the third week of lifting the same comfortable weight: nothing is going wrong, and that is exactly the problem. It grows as sessions pass, faster the more you are already succeeding, and is quieted only by genuine progress; left alone it pushes upward. δ_{conf} is the flinch after a bad run, what makes you re-rack the bar after two failed lifts. It jumps on errors and is quieted by easy successes; left alone it pulls downward. Neither is a goal, and neither knows anything about difficulty levels: the agent observes only which level it chose, whether it was right, and how long it took.

One drive would not do. If the agent is dissatisfied, is the material too easy or too hard? A single scalar deficit registers only "not satisfied" either way — Ashby's requisite variety in concrete form. We treat two antagonistic drives as a design choice rather than a proven minimum, since a single drive with signed updates might recover the distinction and we ran no one-drive ablation. Each drive has setpoint θ and evolves as

$$\delta_{i,t+1} = \delta_{i,t} + \lambda_i m_i - \alpha_i \, g_i[k] \, \mathbf{1}[\text{satiating outcome}] + w_{ij} \, p_i(\delta_i, \theta) \, \mathbf{1}[\delta_j > \theta + \epsilon] \tag{1}$$

reading left to right: basal growth $\lambda_i m_i$ — where m_i is observable mastery for progress and $m_i \equiv 1$ for confidence, which grows at a flat rate (hunger between meals); satiation by $\alpha_i g_i[k]$ when a satisfying outcome arrives at level k; and antagonistic coupling, modulated by the *receiving* drive's own pressure, so a drive at setpoint ignores its counterpart while one under strain responds sharply. Confidence carries one further term the equation above omits for readability: an error raises δ_{conf} by a fixed increment, which is what makes it flinch. Parameters and the exact update, including that increment and the action logits, are in Appendix C.

4.2 The satiation signal, and why it is the crux

$g_i[k]$ decides how much a correct answer at level k counts as progress. Set it wrong and the machinery still works perfectly — it just regulates toward the wrong place. Fix g to a constant and every correct answer, however trivial, quiets the progress drive equally; the agent settles at the easiest level it can find and calls itself satisfied. That is the SSP in one sentence. The learning rule uses only what the agent can see, a running accuracy per level:

$$g_{\text{prog}}[k] \leftarrow (1-\eta) \, g_{\text{prog}}[k] + \eta \cdot 4 \, \text{acc}[k] \, (1 - \text{acc}[k]), \qquad g_{\text{conf}}[k] \leftarrow (1-\eta) \, g_{\text{conf}}[k] + \eta \cdot \text{acc}[k] \tag{2}$$

A level where you get everything right teaches little, and so does one where you get everything wrong. A mastered level therefore yields low g_{prog}, correct answers barely satisfy the progress drive, it keeps growing, and the agent climbs; an impossible level also yields low g_{prog} while errors inflate confidence, and the agent retreats. **No level is ever named as the target — the band is where the dynamics come to rest.** We are precise about what that does and does not claim: the mapping $4a(1 - a)$ encodes a target *accuracy*, and under a Rasch model a target accuracy is a target difficulty gap. So the level identity is emergent while the challenge target is authored, and it would only be fully emergent if the mapping from consequences to satiation were itself learned. Note what is and is not learned: the agent estimates per-level accuracy from experience, but the mapping from accuracy to satiation is hand-authored, so what is learned are the sufficient statistics g consumes rather than the satiation mapping itself. Two properties bound what follows. $4a(1-a)$ peaks at a=0.5, whereas expected gain here is maximized where accuracy is 0.354, so the rule's peak sits on the wrong side of the true optimum however much data it collects. And satiation fires only on correct answers, so the quantity the policy actually receives is $p \cdot \alpha \cdot g$, not g: every comparison we make between a stored g and the expected-gain optimum is therefore one step removed from the operative signal, and an outcome-specific g able to credit informative failures — which this environment rewards but the progress drive ignores — would be the cleaner formulation.

4.3 The environment, and the two things that make it a test

Correctness at level k follows a Rasch model [10], $p = \sigma(1.5(h_{\text{eff}} - k))$. Learning peaks at a moderate stretch above current ability and falls off both ways, and failing at something challenging still teaches, so playing safe is not trivially optimal. Effort accumulates and drags down both effective ability and the capacity to learn; recovery slows sharply past a threshold, so exhaustion cannot be slept off. Resting costs a session and induces mild forgetting. Frustration rises with errors, more so when failing at something previously mastered, and if sustained near a personal threshold for three sessions the student quits for good (Appendix B).

Two details are load-bearing. Fatigue must carry a *deferred* cost — an estimator updating from outcomes already tracks current fatigue for free, so an accumulator earns its keep only if strain that is cheap now is expensive later — and quitting must be reachable, or viability is not a live variable. An earlier version of this environment failed both, which is why it produced an uninterpretable null.

Primary outcome. Every contrast is on **learning gain**, $h_{\text{final}} - h_0$: *latent* ability accumulated over the budget. Terminal fatigue is not penalized directly; it acts through the trajectory, by suppressing how much is learned in each fatigued session. A terminal exam score is a secondary diagnostic only, not dropout-adjusted (Appendix 19).

4.4 Turning the adversity up, rather than picking one setting

The primary axis is ϕ, the severity of non-linear fatigue, over $\{0, 0.3, 0.6, 0.9, 1.2, 1.6\}$. Turning it up makes exhaustion bite, which produces errors, then frustration, then reachable quitting — one knob activates both the non-linearity and the viability pressure. A second axis moves the *peak* of the learning curve mid-episode, leaving any agent that models the curve misspecified in a way re-estimation cannot repair.

Sweeping raises its own problem: at extreme adversity an environment can stop discriminating between good and bad policies. We therefore re-run a quality gate at *every* point — hiding at level 1 must stay unprofitable, the quitting boundary must bite, an oracle policy must beat random. It passes for $\phi \leq 1.2$ and fails at $\phi \geq 1.6$. The gate thresholds were not in the committed text, so **the gated range is an exploratory analytic choice**; we report the crossover with the excluded point included, where it is larger (Appendix D).

4.5 Committing to predictions before looking

A sweep with ten arms and six points offers many places to find a story after the fact, so seven predictions (P1–P7, listed verbatim in Appendix W) were written as plain text and hashed with SHA-256 before any sweep ran: `6e888c7e...bfc084`. Anyone can recompute the digest from `predictions.txt` in the supplement. It is self-attested, not a third-party registry: it makes post-hoc editing detectable, nothing more.

4.6 Arms: each one isolates a suspect

The gap could come from the satiation signal, the action policy, the viability mechanism, or none of them. Each arm removes one candidate so the others can be read:

- **Signal:** Γ`_oracle` (ground-truth g), Γ`_fixed` (constant g), Γ`_naive` — everything else held identical, so any difference is the signal.
- **Policy:** Γ`_argmax` (deterministic action rule) and Γ`_restdrive` (rest driven by drives instead of a frustration gate) — same signal, different channel.
- **Viability:** `rule` and `estimator`, each run *with and without* its rest mechanism, so what a rest guard buys separates from what level selection buys. The estimator is a ten-line agent that tracks ability from outcomes and picks the level its model says is best.

One caveat applies to every arm: the rest mechanism — Γ's strain gate and the baselines' rule alike — normalizes frustration by the profile's private threshold, which a real learner would not disclose. This is not merely a shared advantage: it makes viability easy for everyone and so *deactivates condition (4) by design*. Claims about unobserved state here refer to ability and fatigue, not frustration.

100 profiles $\times$ 10 seeds $\times$ 10 arms $\times$ 6 points = 60,000 episodes. Every profile faces every arm, so contrasts use **paired t-tests on profile-level means** (n=100) with 95% CIs and d_z — an earlier analysis of ours used independent-samples tests here and reported a null that pairing overturns. Because several conclusions rest on the *absence* of an effect, we run TOST equivalence tests against δ=0.05; and because P3 is an *existence* claim over arms and adversity points, we evaluate it with a max-statistic sign-flip permutation test over all cells rather than quoting the winning one.

5 Results

5.1 The sweep, and what degrades

Across the valid range the estimator falls from $+1.132$ to $+0.156$ while Γ_learned moves only from $+0.248$ to $+0.143$: its advantage is a property of the benign regime, not of the method. Dropout stays near zero for arms that can rest and climbs from 16% to 100% for the estimator that cannot, confirming **P1**. **P2** is confirmed as an interaction rather than a ratio of aggregates: per-profile gain-versus-ϕ slopes are -0.808 for the estimator against Γ's -0.070, a paired difference of $+0.737$ (d_z=+4.65). The regulated agent is flatter, not merely lower (Appendix E).

5.2 Decomposing the gap: signal, policy, residual

Everything here is for ϕ=0 and does not transfer across the sweep. The 100 profiles are partitioned once, before any tuning: 1–50 are the tuning pool, of which the first 20 rank candidates, and 51–100 are held out. Every number in this section, in the abstract and in the conclusions comes from the held-out 50 (Appendix P).

A 2×2 over signal quality (constant vs. oracle g) and policy parameterization (hand-chosen vs. searched coefficients) decomposes the gap exactly, interaction included (Table 1). Repeated on time-in-band the shares are 8.7, 29.4, -9.7 and 71.7% — though that metric is conditional on study sessions, and the Γ arms rest far more than the baselines, so it measures level selection when studying rather than regulation over the full budget (Appendix F). The residual is what *the two interventions we ran* leave unexplained; it also absorbs unsearched policy space and the fixed action representation, and is not a proven structural constant. The interaction is negative, which is worth pausing on: a searched policy extracts *less* extra benefit from a perfect signal ($+0.047$) than the hand-chosen one does ($+0.065$). Either the random search failed to co-adapt the policy to the better signal, or six linear coefficients cannot express a control law that exploits it — both readings point at channel capacity rather than at the signal.

The oracle normalizes expected gain by 0.042, its maximum at $l_{r\text{eff}}$=1.0; since profiles draw $l_r \in [0.7, 1.3]$, 31 of 100 clip, flattening g_{prog} where it should discriminate. We flag this as a likely downward bias on the signal term rather than a proven lower bound, since g feeds a closed loop and rescaling need not act monotonically.

Does the coupling do anything? Antagonistic coupling is what distinguishes Γ from two independent thermostats, yet $w_{cp} = -0.05$ and $w_{pc} = +0.05$ are small against drives ranging over $[0, 2]$. Disabling it changes performance and *changes sign*: coupling costs -0.018 at ϕ=0 and buys $+0.024$ and $+0.030$ at ϕ=0.6 and 1.2 (all $p < .001$) — a liability in benign conditions, an asset under adversity, at a magnitude comparable to the whole signal term.

Table 1: Decomposition of the ϕ=0 gap on the 50 held-out profiles. **Key take-away:** under a fixed policy, signal quality is the smaller term; against the model-free estimator the shares become 11.2, 36.6, -3.1, 55.2% — unchanged in ordering (Appendix F).

Component	gain	%
Signal quality (oracle − constant)	+0.065	7.7
Policy coefficients (searched − hand)	+0.211	25.0
Interaction	−0.018	−2.1
Residual	+0.588	69.5
Total gap vs. model-based estimator	+0.846	100.0

5.3 What a signal is worth depends on the policy

The shared coefficients above were optimized against stable signals, so applying them to a noisy learned signal is not a fair test. We repeated the search per signal — five independent searches each, at two budgets, validated on the held-out 50 (Appendix N). At 250 candidates the oracle beats a constant by $+0.047$ (d_z=+1.22) and a learned g is equivalent to a constant ($+0.008$, TOST $p < 10^{-4}$). At 1200 the oracle's advantage is $+0.121$ (CI $[+0.029, +0.213]$, p=.022) and the learned signal's $+0.094$ (p=.062). We are careful not to read a difference in significance as a difference in effect: the direct interaction test, treating the search as the random unit, gives $+0.074$, CI $[-0.026, +0.175]$, p=.11. The larger budget produced the larger *observed* advantage; that the advantage genuinely grows is suggested, not established. Note too that at the profile level the 250-candidate oracle advantage is itself highly significant (d_z=+1.22) — the two frames disagree, and we report both.

The equivalence holds *conditional on these searches*: treating the search as the random unit, the learned-minus-constant difference changes sign across seeds (p=.85, TOST = .18) — inconclusive rather than equivalent as a property of the procedure. An earlier single-search version also produced a non-monotonic ordering, a learned g beating a perfect one, that five searches erase.

The learning rule was aiming at the wrong place. $4a(1-a)$ peaks at accuracy 0.5 while expected gain is maximised at 0.354, which confounds any conclusion that *learning* does not help. With an aligned rule the learned signal reaches $+0.321$ against a constant's $+0.271$ at ϕ=0 (p=4×10^{-34}), where the misaligned rule scored *below* it at $+0.248$. Two qualifications: the advantage vanishes under adversity, and $a^\star$=0.354 comes from the environment's true gain function, so the arm uses privileged knowledge a deployed agent would lack (Appendix J). Learning g does buy something over a constant; what survives is the ordering, since even correctly aimed the signal term is a fraction of the policy term.

5.4 Signal quality, policy noise, and the crossover

P5 is confirmed with a boundary. At ϕ=0 the expected-gain oracle is worth $+0.063$ (d_z=+1.59); from ϕ=0.3 onward the difference falls inside the equivalence bound at every margin from 0.02 to 0.10 — practical equivalence, not a proven null. This corrects an earlier analysis of ours that applied independent-samples tests to this paired design (Appendix G).

P4 is confirmed (d_z from $+0.21$ to $+3.23$), though argmax is not a clean isolation of sampling noise: changing the action rule moves the whole closed loop, and Γ_argmax attains higher signal validity under an identical learning rule. **P6 is confirmed over the gated range but is policy-dependent**: validity for Γ_learned falls from $+0.493$ to $+0.281$ while Γ_argmax holds at $+0.452$. **P7 is refuted**: moving the learning curve's peak mid-episode misspecifies the estimator in a way re-estimation cannot repair, yet the gap *widens* ($-0.181 \rightarrow -0.198$). We first read that as refuting situated information; it does not, since the manipulation corrupts the competitor's model rather than granting Γ a private signal (Appendix R).

P3 is confirmed against the model-based estimator, by $+0.045$ at ϕ=1.2 (CI $[+0.028, +0.062]$), surviving multiplicity correction over all Γ arms and adversity points (p=.0005). Three qualifications withdraw most of its force. It belongs to the deterministic `Γ_argmax` ablation, not the main agent, which scores $+0.143$ against $+0.156$. It sits below our own δ=0.05 threshold. And it does not survive a stronger baseline: an estimator stripped of its objective model scores $+0.275$ at that point.

That ablation narrows a natural objection — the *explicit gain-curve model* is not what wins, since removing it costs the estimator in the benign regime ($+1.132 \to +0.851$) yet *helps* it under adversity, while it stays 1.9 to 3.4 times ahead of Γ throughout. It does not show the estimator carries no task knowledge: it still uses the Rasch model and a hand-coded map from ability to level (Appendix I).

6 Discussion

So: can a homeostatic agent learn g from observable consequences? Partly — it recovers the rank order of g_{prog} (validity $+0.493$ against a $+0.994$ ceiling), though that is a terminal five-point correlation, silent on g_{conf}, on calibration, and on the operative $p \cdot \alpha \cdot g$. And can it use that signal? Only marginally: an oracle buys under a tenth of the distance to a simple estimator while a rigid policy buys over three times more.

Viability was our most confident claim and it did not survive. It rested on comparing Γ against an estimator with *no* rest mechanism — only one side could rest. Disable rest on both and the estimator hits 100% dropout against Γ's 93.5%; worse, the *rule* baseline stripped of rest reaches only 55% at $\phi{=}1.2$, *more* viable than Γ without rest. Nor is the deficit the cost of resting more: matching the estimator's rest count to Γ's costs it -0.307 yet leaves it $3.3\times$ ahead (Appendix K). Viability is secured by *a rest mechanism*, and a two-line heuristic secures more of it than the drives do. But the drives are not bystanders, and an earlier framing of ours implied they were. Γ's rest utility reads the confidence deviation, so the mechanism is a drive-conditioned policy action rather than an independent guard: strip that term and leave only the frustration gate, and dropout rises from 1.0% to 20.2% at $\phi{=}1.2$ (paired $p < .001$). The accurate statement is that a rest *action* is necessary and a hand-coded gate is nearly sufficient, while the drives sharpen when it fires.

None of this is comfortable for a framework that names five conditions under which a homeostatic agent should be the right tool, so: does this domain satisfy them? Condition (3) holds cleanly and delivers the predicted signature. Condition (5) holds only *partially*, since the environment dissociates correctness from learning, so the agent never observes an objective signal for the quantity it should regulate toward. Condition (1) holds *cheaply*, a one-line guard erasing a reachable failure, and (2) only nominally, since the estimator carries no confidence drive and still wins. Condition (4) fails outright: an external observer of level and correctness alone reconstructs effective ability to within 0.32 at $\phi{=}0$, where levels sit 1.0 apart — though that measures ability in the benign regime, not the fatigue, frustration and threshold that matter under adversity, and the threshold is disclosed to everyone by design.

That asymmetry is the crux. Conditions (1), (2), (3) and (5) establish only that regulation is *needed*; (4) establishes that the agent must be the one doing it. It is load-bearing and we treated all five as equals — though the measurement shows only that *this* environment grants no private information, not that Γ would exploit any it had.

A sharper reading deserves stating. Under a Rasch model the optimal difficulty is a closed function of ability, so "learning the policy" would converge on reimplementing the estimator — on which reading the channel is no bottleneck but evidence that this problem never needed a regulator. We concede that *for terminal objectives*. What stops us generalising is a sixth condition we suspect explains our result better than the other five: homeostasis is a *maintenance* formalism with no horizon, while our task has a terminal objective whose gain can be banked. **The objective must be the maintenance itself, not a terminal payoff that maintenance merely enables.** Replacing it with one that cannot be banked — mean readiness, averaged over every session — does reverse the ordering, but for one arm under a conjunction of three conditions, and Γ`_learned` never overtakes the model-free estimator (Appendix M), so we offer it as *suggested*. It could have failed, and a first version did: lengthening the horizon alone changed nothing. Duration was never the variable. Bankability was. (Appendix O lists all eight withdrawn claims.)

7 Limitations

The crossover is small, belongs to the deterministic ablation, and does not survive the stronger baseline. **Signal validity is incompletely operationalised**: only g_{prog}, as a terminal five-point correlation, with g_{conf} unvalidated and no measure of the operative $p \cdot \alpha \cdot g$. ϕ **is a compound knob**, so the sweep cannot isolate non-linearity, and **policy tuning was a random search**, so the residual is an upper bound on what is structural. **The two sides are not representationally matched**: Γ's channel is six linear coefficients over two drive deviations and never observes the current level, while the estimator tracks ability and applies a non-linear argmax over levels. A channel that cannot express an effective control law cannot profit from a better signal, so "the policy is the bottleneck" is established for *this* channel and is partly a statement about its capacity; testing a richer policy class is the first thing we would run next. **Researcher degrees of freedom remain**: a self-attested hash, uncommitted gate thresholds, a post-hoc sixth condition, and untested conditions (2) and (4). Finally, every experiment here runs in seconds on a laptop with no GPU, and no language model participates in the agent, environment, baselines or analysis; LLMs were used only as coding and writing assistants (Appendix U).

8 Future Work

The decomposition sets the priority: with most of the gap unexplained by signal or coefficients, the next step is to **learn the policy**, not the signal — starting with a richer policy class than six linear coefficients, since our own channel may be too impoverished to profit from a better signal, and searching signal and policy jointly, since matched tuning shows they must be co-adapted. Conditions (2) and (4) need dedicated tests: a one-drive ablation, and an experiment granting Γ a private internal signal withheld from the estimator. A further question follows from our own null: if a learned g barely beats a constant, the culprit may be that it *keeps* learning, since a constant has zero variance while a signal from six visits per level injects noise indefinitely. Would a **convergence-stopped** signal keep the bias reduction while shedding the variance?

Most consequential is allostasis. Homeostasis corrects after deviation; allostasis anticipates it [2]. The jump demands a **world model**: a learned predictor of environmental dynamics and of how the agent's actions propagate through them. What separates it from a self-model is that the regularities live *in the world* — harder material produces errors at a rate set by the gap to ability, effort compounds non-linearly, recovery has a threshold. The agent authors none of these laws and must infer them; its drives are merely where they register. Knowing your own state predicts nothing unless you also know how the world will act on it: the system to model is the coupled pair, not the agent alone.

9 Impact Statement

Agents that pair a rest mechanism with learned internal needs could yield tutoring systems that respect a learner's cognitive and emotional limits — though much of the protective work is done by the rest mechanism, which non-homeostatic baselines exploit too. The mechanism inverts cleanly: what models frustration to avoid it can model engagement to maximise it, a machine that learns when to intervene to keep someone hooked. The distinction between regulating *for* wellbeing and *to exploit* attention lies not in the mechanism but in whose homeostasis is regulated and to what end. Our environment hands every agent the learner's private frustration threshold; a deployed system would have to infer it, and one accurate enough to protect a learner is accurate enough to exploit one. Mitigation is institutional rather than technical: transparency about which objective the regulator serves, and who chose it.

10 Conclusion

Can a homeostatic agent learn its satiation signal from observable consequences and use it to regulate toward an external goal? Partly, and less than we expected: at $\phi{=}0$ an expected-gain oracle is worth 7.7% of the gap to a simple estimator against 25.0% for six policy coefficients. Diagnosing why exposed our framework as under-specified — of five conditions only one, that the agent hold what an outside observer cannot, makes the architecture necessary rather than applicable. Knowing what you need is not what limits you: the open problem is the channel.

References

[1] **Cannon, W.B.** (1932). *The Wisdom of the Body*. New York: W.W. Norton.

[2] **Sterling, P. & Eyer, J.** (1988). Allostasis: A new paradigm to explain arousal pathology. In S. Fisher & J. Reason (eds.), *Handbook of Life Stress, Cognition and Health*, pp. 629–649. New York: Wiley.

[3] **Keramati, M. & Gutkin, B.S.** (2014). Homeostatic reinforcement learning for integrating reward collection and physiological stability. *eLife*, 3, e04811.

[4] **Ashby, W.R.** (1952). *Design for a Brain*. London: Chapman & Hall.

[5] **Dayan, P.** (1993). Improving generalization for temporal difference learning: The successor representation. *Neural Computation*, 5(4), 613–624.

[6] **Gershman, S.J., Moore, C.D., Todd, M.T., Norman, K.A., & Sederberg, P.B.** (2012). The successor representation and temporal context. *Neural Computation*, 24(6), 1553–1581.

[7] **Lehnert, L. & Littman, M.L.** (2019). Successor features combine elements of model-free and model-based reinforcement learning. *Journal of Machine Learning Research*, 20(1), 1–53.

[8] **Zubek, R.** (2010). Needs-Based AI. In A. Lake (ed.), *Game Programming Gems 8*, pp. 231–238. Charles River Media.

[9] **Vygotsky, L.S.** (1978). *Mind in Society: The Development of Higher Psychological Processes*. Cambridge, MA: Harvard University Press.

[10] **Rasch, G.** (1960). *Probabilistic Models for Some Intelligence and Attainment Tests*. Copenhagen: Danish Institute for Educational Research.

A Extended related work

The gap we identify matters beyond homeostatic RL. The successor representation literature [5–7] showed that agents benefit from modelling how the environment affects future states — which our residual term identifies as the necessary next step, and which motivates the world-model direction of our future work.

A critique of the whole family is worth recording. Several recent discussions (2025–2026) raise what may be called a *rule-relocation* objection: in homeostatic architectures the setpoint is itself a norm imposed by the designer, so normativity is relocated from the reward function to the setpoint rather than eliminated. The Satiation Signal Problem is a special case, g being the relocated norm. We flag this as an open conceptual issue rather than attributing it to a specific source, since the argument appears in discussion rather than in a single citable formulation.

B Environment specification

Ability follows a Rasch model [10], $p = \sigma(1.5(h_{\text{eff}} - k))$. Learning gain per exercise is

$$\Delta h = 0.08 \exp\left(-\frac{(\text{gap} - 0.5)^2}{0.5}\right) \cdot c \cdot \text{lr}_{\text{eff}}, \qquad \text{gap} = k - h_{\text{eff}},\ c \in \{1.0, 0.3\}$$

so failing at something challenging still teaches. Effort accumulates as $e \leftarrow 0.95e + \ell/20$ with ℓ the response latency, normalized by E_{ref}=3.0. Fatigue acts on two channels:

$$h_{\text{eff}} = \max(0.5,\ h - \phi e_n^{1.5}), \qquad \text{lr}_{\text{eff}} = \text{lr} \cdot \max(0.15,\ 1 - 0.7\phi e_n^{1.5})$$

The second is essential: without it, fatigue merely widens the difficulty gap and therefore moves the agent *toward* the bell curve's peak, so a fatigued student would learn *more* per session — an artefact that renders any fatigue sweep uninterpretable. Rest multiplies effort by 0.7, or 0.93 once e_n exceeds 1.3 (hysteresis), and costs 0.01 of ability. Frustration rises 0.25 per error, 0.3 more when failing at a mastered level, falls 0.35 on accessible successes, halves on rest, and triggers absorbing dropout at $0.95\times$ threshold for three consecutive sessions. Profiles draw $h_0 \in [1.5, 3.5]$, lr $\in [0.7, 1.3]$, threshold $\in [1.8, 3.0]$.

C Agent specification

Configuration: λ_p=0.15, λ_c=0.02, θ=0.7, α_p=α_c=0.3, w_{cp}=$-$0.05, w_{pc}=0.05, η=0.3, β=0.2.

```
def select(self, rng, env):
    dp = self.d_prog - theta                        # progress deviation
    dc = self.d_conf - theta                        # confidence deviation
    fn = self.frust / self.profile["umbral"]        # normalized frustration
    strain = max(0.0, fn - 0.7) / 0.3               # rest gate

    if rest_mode == "gated":                        # default arm
        l_rest = 2*max(0.0, dc)*strain + 4*strain
    elif rest_mode == "none":                       # viability ablation
        l_rest = -1e9
    else:                                           # drive-driven ablation
        l_rest = 3.0*max(0.0, dc) + 2.0*strain - 1.2

    logits = [c0*dp + c1*dc,                        # UP
              c2*(abs(dp) + abs(dc)) + c3,          # STAY
              c4*dc + c5*dp,                        # DOWN
              l_rest]                               # REST
    return ACTIONS[argmax(logits) if action_mode=="argmax"
                   else sample_softmax(logits, rng)]
```

In the default (gated) mode every term of `l_rest` is multiplied by `strain`, so the drives do not participate in the rest decision: along the viability dimension the agent is as reactive as the baselines it is compared against. This is why the viability claim in Section 6 is restricted to the `restdrive` arm.

D Quality gate across the sweep

Table 2: Gate criteria at each sweep point. **Key take-away:** the environment stops discriminating at ϕ=1.6, where the level-1 trap dissolves.

ϕ	always1	always5 drop	random	oracle	margin
0.0	+0.010	100%	+0.596	+1.139	+0.544 (PASS)
0.3	+0.035	100%	+0.307	+0.868	+0.561 (PASS)
0.6	+0.073	100%	+0.182	+0.533	+0.351 (PASS)
0.9	+0.108	100%	+0.133	+0.406	+0.273 (PASS)
1.2	+0.144	100%	+0.107	+0.357	+0.250 (PASS)
1.6	+0.162	100%	+0.088	+0.316	+0.228 (FAIL)

Only the first criterion fails at ϕ=1.6 (+0.162 against a 0.15 cutoff); the oracle margin remains healthy.

Sensitivity to the exclusion. At ϕ=1.2, Γ_argmax minus estimator is +0.045, CI [+0.028, +0.062], d_z=+0.52; at the excluded ϕ=1.6 it is +0.089, CI [+0.078, +0.101], d_z=+1.54. Including the point strengthens the claim, so the exclusion is conservative.

E Full main sweep (all ten arms)

Table 3: Learning gain / % dropout for every arm at every valid sweep point.

Arm	ϕ=0.0	ϕ=0.3	ϕ=0.6	ϕ=0.9	ϕ=1.2
Γ_learned	+0.248/0%	+0.148/0%	+0.138/0%	+0.146/1%	+0.143/1%
Γ_argmax	+0.256/0%	+0.184/0%	+0.179/0%	+0.199/0%	+0.201/0%
Γ_restdrive	+0.363/4%	+0.186/7%	+0.165/12%	+0.161/15%	+0.154/16%
Γ_fixed	+0.271/0%	+0.142/0%	+0.148/0%	+0.151/0%	+0.147/0%
Γ_naive	+0.269/0%	+0.164/0%	+0.139/0%	+0.150/0%	+0.147/0%
Γ_oracle	+0.335/0%	+0.149/0%	+0.151/0%	+0.150/0%	+0.145/0%
rule	+0.591/0%	+0.413/0%	+0.270/0%	+0.220/0%	+0.199/0%
rule_norest	+0.595/0%	+0.417/1%	+0.278/13%	+0.229/38%	+0.207/55%
estimator	+1.132/0%	+0.700/0%	+0.360/0%	+0.228/0%	+0.156/0%
estimator_norest	+1.108/16%	+0.612/63%	+0.317/94%	+0.207/100%	+0.156/100%

F Additive decomposition

Table 4: Components of the ϕ=0 gap. **Left pair: the held-out estimate used in the abstract, body and conclusions** (coefficients ranked on 20 scoring profiles, evaluated on 50 never used for tuning). **Right pair: the superseded partly in-sample estimate**, retained only to show what leakage costs. **Key take-away:** leakage inflates the policy term by under three points and leaves the ordering unchanged.

Component	held-out	%	in-sample	%
Signal quality (oracle − constant, hand policy)	+0.065	7.7	+0.063	7.4
Policy coefficients (re-tuned − hand, constant g)	+0.211	25.0	+0.238	27.7
Interaction	−0.018	−2.1	−0.047	−5.5
Residual (estimator − best Γ)	+0.588	69.5	+0.606	70.5
Total gap vs. model-based estimator	+0.846	100.0	+0.860	100.0
held-out, vs. model-free estimator (total +0.576): 11.2, 36.6, −3.1, 55.2%				

G Signal-quality contrast across adversity

Table 5: P5: oracle versus constant g, paired over 100 profiles.

ϕ	oracle	constant	diff	95% CI	p	d_z
0.0	+0.335	+0.271	+0.063	[+0.055, +0.071]	6.4e-29	+1.59
0.3	+0.149	+0.142	+0.007	[+0.003, +0.011]	6.0e-04	+0.35
0.6	+0.151	+0.148	+0.003	[+0.000, +0.006]	2.0e-02	+0.24
0.9	+0.150	+0.151	-0.001	[-0.004, +0.001]	2.7e-01	-0.11
1.2	+0.145	+0.147	-0.002	[-0.004, -0.000]	3.2e-02	-0.22

Table 6: TOST equivalence p-values for oracle minus constant across candidate margins. **Key take-away:** from ϕ=0.3 onward the difference is inside every margin tested, so the conclusion does not depend on δ.

ϕ	diff	δ=0.02	δ=0.03	δ=0.05	δ=0.075	δ=0.10
0.0	+0.063	1.000	1.000	0.999	0.002	$<$.001
0.3	+0.007	$<$.001	$<$.001	$<$.001	$<$.001	$<$.001
0.6	+0.003	$<$.001	$<$.001	$<$.001	$<$.001	$<$.001
0.9	$-$0.001	$<$.001	$<$.001	$<$.001	$<$.001	$<$.001
1.2	$-$0.002	$<$.001	$<$.001	$<$.001	$<$.001	$<$.001

H Policy and crossover contrasts

Table 7: Paired contrasts over 100 profiles. Left: argmax minus softmax (P4). Right: Γ_argmax minus the model-based estimator (P3).

	P4: argmax − softmax			P3: Γ_argmax − estimator		
ϕ	diff	95% CI	d_z	diff	95% CI	d_z
0.0	+0.008	[+0.000, +0.016]	+0.21	-0.876	[-0.908, -0.843]	-5.29
0.3	+0.036	[+0.029, +0.043]	+1.00	-0.517	[-0.562, -0.472]	-2.27
0.6	+0.041	[+0.035, +0.048]	+1.25	-0.181	[-0.216, -0.146]	-1.02
0.9	+0.053	[+0.048, +0.058]	+1.98	-0.029	[-0.054, -0.004]	-0.23
1.2	+0.058	[+0.054, +0.061]	+3.23	+0.045	[+0.028, +0.062]	+0.52

I Objective-model ablation

Table 8: Learning gain with and without the objective model. **Key take-away:** removing privileged knowledge of the learning curve costs the estimator in the benign regime but *helps* it under adversity — and it still beats the homeostatic agent everywhere.

ϕ	estimator	estimator, no objective model	Γ_learned
0.0	+1.132	+0.851	+0.248
0.6	+0.360	+0.474	+0.138
1.2	+0.156	**+0.275**	+0.143

J Aligned learning rule

The learned rule $4a(1-a)$ is symmetric about a=0.5, while expected gain in this environment peaks at a difficulty gap of 0.40, where accuracy is 0.354. The rule's target is therefore unreachable by construction, independently of estimation noise. We re-ran with $g_{\text{prog}} \leftarrow (1-\eta)g_{\text{prog}} + \eta\exp(-(a-a^{\star})^2/2\sigma^2)$, $a^{\star}$=0.354, σ=0.18 — the same observable input, aimed at the right place.

Table 9: Learning gain by signal under the hand-chosen policy, 100 paired profiles. **Key take-away:** correcting the functional form moves the learned signal from below a constant to above it, recovering most of the oracle's advantage — so the earlier "learning buys nothing" result was partly an artefact of the rule's target.

ϕ	constant	learned ($4a(1-a)$)	learned (aligned)	aligned − constant	p
0.0	+0.271	+0.248	**+0.321**	$+0.050$	4×10^{-34}
0.6	+0.148	+0.138	+0.147	-0.001	.37
1.2	+0.147	+0.143	+0.147	$+0.000$	.72

For reference the expected-gain oracle scores $+0.335$ under the same policy, so the aligned learned signal captures roughly 79% of the achievable signal benefit at ϕ=0. Under adversity every signal converges, consistent with the equivalence reported in the main text.

K Rest-budget ablation

Γ spends 9.1 of 40 sessions resting at ϕ=0 against the estimator's 1.1, so a fifth of its budget buys no study. With cumulative gain as the primary outcome, that alone could explain part of the deficit. We therefore forced the estimator to rest as often as Γ, spread evenly across the episode.

Table 10: Learning gain when the estimator is forced to match Γ's rest count. **Key take-away:** resting is expensive in the benign regime and *profitable* under adversity, where the estimator was under-resting — either way it does not explain the gap.

ϕ	Γ_learned	estimator	estimator $+9$ rests	cost of resting
0.0	+0.248	+1.132	+0.824	-0.307
0.6	+0.138	+0.360	+0.587	$+0.228$
1.2	+0.143	+0.156	+0.298	$+0.142$

At ϕ=0 the forced rests cost the estimator -0.307, roughly a third of its lead — yet it still outscores Γ by $3.3\times$. Under adversity the forced rests *help* it, by $+0.228$ and $+0.142$, which says the estimator's hand-coded rule was resting too little rather than that Γ rests too much. Rest accounting is therefore a real but insufficient explanation of the deficit, and it does not rescue the regulated agent.

L Viability ablation

Table 11: Dropout $\%$ with and without a rest mechanism on *both* sides. **Key take-away:** without rest both architectures collapse, the regulated one slightly later; with rest both are safe. Viability tracks the presence of a rest mechanism, not of homeostatic drives.

	no rest mechanism		with rest mechanism	
ϕ	Γ_learned	estimator	Γ_learned	estimator
0.0	6.4%	16.4%	0.0%	0.0%
0.6	81.5%	94.5%	0.4%	0.0%
1.2	93.5%	100.0%	1.0%	0.0%

M Maintenance-objective experiment

The primary task has a terminal objective: accumulate ability by session 40. Gain can be banked, so more is always better. To test whether homeostatic regulation fares better under a *maintenance* objective we replace it with **mean readiness** — $\sigma(1.5(h_{\text{eff}} - 3))$, the score the student would obtain if the exam were held that session, averaged over the horizon, with zero credit for every session after quitting. Readiness cannot be banked: fatigue costs continuously rather than only at a deadline.

Table 12: Mean readiness over 60 profiles $\times$ 6 seeds. **Key take-away:** the ordering reverses only under the conjunction of a non-bankable objective, a long horizon, and severe adversity — at the paper's own setting (top row) the estimators still win.

ϕ	horizon	Γ_learned	Γ_argmax	estimator	estimator (no obj.)	rule
0.6	40	0.158	0.141	0.203	**0.231**	0.178
0.6	100	0.110	0.103	0.122	**0.168**	0.115
0.6	200	0.119	0.134	0.102	**0.160**	0.097
0.6	400	0.145	0.196	0.120	**0.210**	0.122
1.2	40	0.099	0.084	0.102	**0.114**	0.098
1.2	100	0.054	0.049	0.055	**0.061**	0.053
1.2	200	0.038	0.040	0.039	**0.043**	0.039
1.2	400	0.041	**0.080**	0.035	0.049	0.037

Paired contrasts for Γ_argmax, n=60 profiles: at ϕ=0.6, H=40 it trails the estimator by -0.062 (d_z=-1.07) and the model-free estimator by -0.090 (d_z=-1.29). At ϕ=0.6, H=400 it leads the estimator by $+0.076$ (d_z=+2.09) but trails the model-free one by -0.014. At ϕ=1.2, H=400 it leads both, by $+0.044$ (d_z=+0.73) and $+0.031$ (d_z=+0.61), all $p < 10^{-4}$.

A failed first attempt. We first tested the hypothesis by extending the horizon alone while keeping cumulative gain as the objective. That test failed: no baseline ever quit, the estimator's advantage *grew* with horizon, and Γ's survival fell to 85% at 400 sessions. Lengthening a horizon does not make an accumulative objective a maintenance objective, which is the distinction the sixth condition turns on. We report the failed version because it is what identified the right variable.

N Matched-tuning results

Five independent search seeds at each budget, all validated on the 50 profiles never used for tuning.

Table 13: Held-out gain per search seed. **Key take-away:** at the smaller budget the learned-minus-constant difference changes sign across searches, which is why the equivalence claim is conditioned on these searches rather than asserted for the tuning procedure.

Budget	Signal	s1	s2	s3	s4	s5	mean	sd
250	constant	.464	.458	.473	.412	.554	.472	.051
250	learned	.439	.455	.599	.471	.438	.480	.068
250	oracle	.509	.569	.499	.447	.570	.519	.052
1200	constant	.411	.466	.478	.574	.505	.487	.060
1200	learned	.545	.680	.527	.579	.572	.581	.059
1200	oracle	.656	.569	.604	.631	.579	.608	.036
estimator: $+1.132$*; estimator without objective model:* $+0.851$								

Profile-level (n=50 profiles, conditional on these searches): at 250, learned $-$ constant $= +0.008$, CI $[-0.002, +0.018]$, TOST $p < 10^{-4}$; oracle $-$ constant $= +0.047$, CI $[+0.036, +0.058]$, d_z=+1.22. **Search-level** (n=5 searches, the tuning procedure as the random unit): at 250, learned $-$ constant p=.85 with TOST = .18 (inconclusive) and oracle $-$ constant p=.050; at 1200, oracle $-$ constant $= +0.121$, CI $[+0.029, +0.213]$, p=.022, and learned $-$ constant $= +0.094$, p=.062.

O Claims we withdrew, and what caught them

Table 14: Every claim of ours that a stricter protocol overturned. **Key take-away:** each fell to one specific methodological choice, which is the transferable part.

Claim	What overturned it
Signal quality does not reach behaviour at all	Paired tests on the paired design: $+0.063$, d_z=$+1.59$ at ϕ=0
A learned g beats a perfect one	Five independent searches: ordering monotone, oracle above constant $5/5$
A learned g is counterproductive versus a constant	Matched per-signal tuning
A learned g is equivalent to a constant	Search-level uncertainty: TOST $=.18$, inconclusive
Signal quality is simply the smallest term	Its value is established only under a larger policy search
Homeostasis uniquely buys viability	Rest removed from both sides; `rule_norest` is more viable than Γ without rest
Homeostasis integrates stopping into the same machinery	The default rest logit is frustration-gated
P7 refutes the value of situated information	The manipulation corrupts the competitor's model instead

P Profile usage for every analysis

The 100 profiles are partitioned once, before any tuning, and the partition never changes.

Table 15: Which profiles each analysis uses, and whether they participated in candidate selection. **Key take-away:** every number quoted in the abstract, body and conclusions comes from profiles that never participated in selecting a coefficient.

Analysis	Scoring (selection)	Evaluation	Leakage
Main sweep (Sec. 5.1)	—	all 100	none (no tuning)
Decomposition (Sec. 5.2)	profiles 1–20	profiles 51–100	none
Matched tuning, 250 (Sec. 5.3)	profiles 1–20	profiles 51–100	none
Matched tuning, 1200 (Sec. 5.3)	profiles 1–20	profiles 51–100	none
Ablations (Sec. 5.4, 6)	—	all 100	none (no tuning)
Shape shift (Sec. 5.4)	—	all 100	none (no tuning)
Reconstruction (Sec. 6)	—	40 profiles	none (no tuning)
Maintenance (Sec. 6)	—	60 profiles	none (no tuning)
Superseded shared search (App. Q)	30 profiles	all 100	*partly in-sample*

Scoring uses 3 of the 10 seeds; evaluation always uses all 10. Only the last row involves leakage, and no number reported outside that row's own table derives from it.

Q Policy coefficient search

The four action logits are linear in d_p, d_c with six free coefficients; the original values $(4, -3, -3, 1.5, 4, -3)$ were chosen by hand. Candidates are sampled from $c_0, c_4 \sim U(0, 10)$; $c_1, c_5 \sim U(-10, 2)$; $c_2 \sim U(-10, 0)$; $c_3 \sim U(-2, 5)$.

The protocol supporting the paper searches independently per signal, ranks candidates on the 20-profile scoring subset, and validates on the 50 strictly held-out profiles, with five search seeds at each budget (Appendix P).

The table below is a superseded earlier run in which a single shared search was ranked on 30 profiles and validated on all 100, including those 30. We retain it only to price the leakage: it inflates the policy term from $+0.211$ to $+0.238$ (25.0% to 27.7% of the gap) and leaves the ordering unchanged. No headline number comes from it.

Table 16: Effect of coefficient re-tuning at ϕ=0, shared search, validated on 100 profiles including the 30 used for scoring (partly in-sample; see Section 5.3 for the held-out version).

Arm	hand-chosen	re-tuned	Δ
Γ_oracle	+0.335	+0.526	+0.191
Γ_fixed	+0.271	+0.509	+0.238
Γ_learned	+0.248	+0.266	+0.018
estimator: $+1.132$			

500 ## R Shape-shift results and quality gate

Table 17: Peak of the learning curve moved at session 20; ϕ=0.6; 100 paired profiles. **Key take-away:** all four points pass the gate, and the estimator's lead grows as its own model is corrupted.

peak	Γ_argmax	Γ_oracle	estimator	gap	d_z	gate margin
0.5	+0.179	+0.151	+0.360	−0.181	−1.02	+0.370 (PASS)
1.0	+0.145	+0.110	+0.345	−0.200	−1.19	+0.316 (PASS)
1.5	+0.114	+0.068	+0.324	−0.210	−1.32	+0.294 (PASS)
2.0	+0.103	+0.057	+0.301	−0.198	−1.26	+0.280 (PASS)

501 ## S Complete sweep metrics

Table 18: Time-in-band, conditional on study sessions.

Arm	ϕ=0.0	ϕ=0.3	ϕ=0.6	ϕ=0.9	ϕ=1.2
Γ_learned	0.271	0.350	0.519	0.610	0.644
Γ_argmax	0.252	0.340	0.541	0.670	0.715
Γ_restdrive	0.289	0.343	0.519	0.578	0.597
Γ_fixed	0.229	0.354	0.571	0.637	0.656
Γ_naive	0.250	0.341	0.498	0.610	0.648
Γ_oracle	0.279	0.400	0.599	0.647	0.660
rule	0.364	0.409	0.498	0.612	0.670
rule_norest	0.365	0.414	0.510	0.613	0.662
estimator	0.805	0.780	0.592	0.459	0.381
estimator_norest	0.796	0.749	0.589	0.447	0.371

Table 19: Exam score on effective ability. **Key take-away:** the exam ordering does not always follow learning gain — at ϕ=0.6 the rule scores above the estimator (0.087 vs. 0.084) while trailing it on gain (+0.270 vs. +0.360), and the inversion recurs at ϕ=0.9 — which is why gain is the primary outcome and this table a diagnostic only.

Arm	ϕ=0.0	ϕ=0.3	ϕ=0.6	ϕ=0.9	ϕ=1.2
Γ_learned	0.446	0.205	0.085	0.041	0.031
Γ_argmax	0.448	0.192	0.072	0.038	0.028
Γ_restdrive	0.484	0.169	0.055	0.028	0.024
Γ_fixed	0.454	0.164	0.047	0.028	0.023
Γ_naive	0.454	0.189	0.071	0.033	0.027
Γ_oracle	0.474	0.150	0.039	0.025	0.023
rule	0.556	0.251	0.087	0.039	0.026
rule_norest	0.557	0.250	0.083	0.035	0.024
estimator	0.716	0.338	0.084	0.034	0.024
estimator_norest	0.708	0.290	0.073	0.031	0.025

Table 20: Mean REST sessions out of 40. **Key take-away:** the regulated arms spend much of the budget resting — the price of the viability they buy.

Arm	ϕ=0.0	ϕ=0.3	ϕ=0.6	ϕ=0.9	ϕ=1.2
Γ_learned	9.1	8.0	6.7	6.2	6.0
Γ_argmax	8.8	6.5	4.5	3.4	3.0
Γ_restdrive	5.0	4.7	4.4	4.2	4.2
Γ_fixed	6.7	5.7	5.0	5.0	5.0
Γ_naive	7.3	6.7	5.9	5.6	5.6
Γ_oracle	6.3	5.7	5.1	5.1	5.1
rule	0.1	0.3	0.8	1.5	2.0
rule_norest	0.0	0.0	0.0	0.0	0.0
estimator	1.1	2.4	4.0	5.0	5.4
estimator_norest	0.0	0.0	0.0	0.0	0.0

Table 21: Satiation-signal validity. **Key take-away:** validity degrades with adversity for the learned arm but not the oracle.

Arm	ϕ=0.0	ϕ=0.3	ϕ=0.6	ϕ=0.9	ϕ=1.2
Γ_learned	+0.493	+0.415	+0.351	+0.303	+0.281
Γ_argmax	+0.550	+0.443	+0.505	+0.456	+0.452
Γ_restdrive	+0.441	+0.488	+0.470	+0.431	+0.397
Γ_fixed	—	—	—	—	—
Γ_naive	—	—	—	—	—
Γ_oracle	+0.994	+0.861	+0.896	+0.920	+0.909
rule	—	—	—	—	—
rule_norest	—	—	—	—	—
estimator	—	—	—	—	—
estimator_norest	—	—	—	—	—

T Full sweep figure

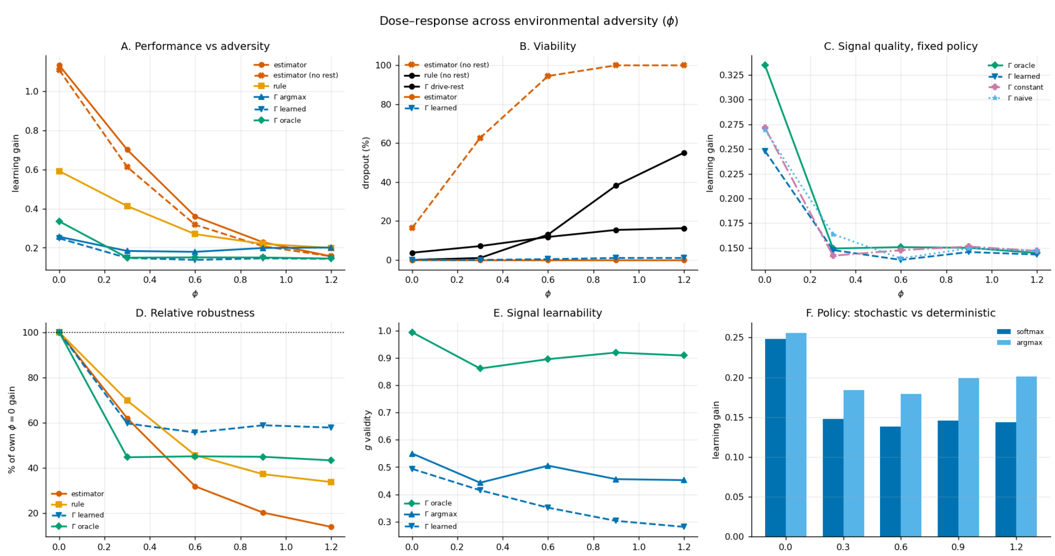

Figure 1: Six-panel summary, Okabe-Ito colourblind-safe palette with distinct markers and line styles so the panels remain legible in greyscale. **A**: performance across adversity. **B**: viability. **C**: signal quality under the fixed hand policy. **D**: robustness normalized to each arm's own ϕ=0 value. **E**: signal learnability. **F**: stochastic versus deterministic policy.

U Compute resources and LLM usage

All experiments run on a commodity laptop with no GPU; the main sweep of 60,000 episodes takes 31.4 s single-threaded and a single decision costs 14 μs; seed derivation and the reproduction entry point are in Appendix V.

We disclose LLM usage for transparency, although it is not a component of the core method: no language model participates in the agent, the environment, the baselines, or the analysis, all of which are explicit numerical simulations. LLMs served as engineering assistants in two capacities — code assistance, via Devin as an IDE together with Kimi 3 and DeepSeek v4 Pro, and Claude Opus 5 and Claude Fable 5; and writing and editing assistance, via Claude Opus 5 and Claude Fable 5. All experimental design decisions, the committed predictions, the statistical procedures, and every interpretation are the authors' own, and all reported numbers were produced by executing the released notebook.

V Reproducibility

Ten seeds are derived from master seed `20260820` via SHA-256, each mapped to the next prime above $10007 + (\text{digest} \bmod 900000)$; profiles come from a separate fixed seed. The released notebook reproduces every reported number and asserts the prediction hash before running. Dependencies are `scipy` and `matplotlib`.

W Preregistered prediction text (verbatim)

The authoritative artifact is `predictions.txt` in the supplement, whose SHA-256 digest is `6e888c7e46e1dd306b6adc601a45cc08ff914a24d2f474a913a1ff2907bfc084`. It contains one prediction per line in sorted key order, joined with newlines. Below the same text is shown with display-only wrapping; continuation lines are indented and are *not* part of the hashed string.

```
P1: bayesian_norest (estimacion sin gestion de viabilidad) colapsa al subir
    phi: dropout alto, gain al piso.
P2: Los brazos Gamma se degradan proporcionalmente menos que el bayesiano a
    lo largo del barrido de phi.
P3: Existe phi* dentro del rango valido de la puerta de calidad donde algun
    brazo Gamma supera al bayesiano en gain.
P4: gamma_learned_argmax supera a gamma_learned en todos los puntos validos
    del barrido.
P5: Si la calidad de g se transmite a la conducta, gamma_oracle se separa de
    gamma_no_learning en algun punto del barrido.
P6: g_validity de gamma_learned decrece monotonamente al subir phi.
P7: Bajo desplazamiento del PICO de la campana de aprendizaje (cambio de
    forma, no de escala), los brazos Gamma ganan terreno relativo frente al
    bayesiano, que queda mal especificado y no puede absorberlo re-estimando h.
```

X Source code

Self-contained: no external dataset, no pretrained model, only `scipy` and `matplotlib` beyond the standard library. **Naming note:** the code predates the paper's terminology — `BayesianAgent` is the paper's *estimator*, `BayesianNoObjAgent` the estimator without an objective model, `ReglaAgent` the *rule*, `gamma_no_learning` the *constant-g* arm; `nivel`, `ganancia` and `descansar` are level, gain and rest.

X.1 Seeds and determinism

```
import hashlib, math, random
from collections import defaultdict

NIVELES=[1,2,3,4,5]
PRESUPUESTO=40
E_REF=3.0

def _is_prime(n):
```

```
    if n<2: return False
    if n in (2,3,5,7): return True
    if n%2==0 or n%3==0: return False
    i=5
    while i*i<=n:
        if n%i==0 or n%(i+2)==0: return False
        i+=6
    return True
def _next_prime(n):
    if n<=2: return 2
    c=n|1
    while not _is_prime(c): c+=2
    return c
def derive_seeds(master,n=10):
    s=[]
    for i in range(n):
        d=hashlib.sha256(f"{master}:run_{i}".encode()).hexdigest()
        s.append(_next_prime(10007+int(d,16)%900000))
    return s

def rasch_p(h,nivel): return 1.0/(1.0+math.exp(-1.5*(h-nivel)))
PEAK_MODEL=0.5   # peak assumed by any agent that models the curve (misspecifiable)
def bell_curve(gap,peak=PEAK_MODEL): return 0.08*math.exp(-((gap-peak)**2)/0.5)
```

X.2 Environment

```
class StudyEnv:
    """v4: fatiga normalizada + histeresis + shift de regimen + examen en h_eff."""
    def __init__(self, h0, lr_mult=1.0, phi=0.0, lambda_olv=0.01,
                 hysteresis=False, e_crit=1.3, shift_mult=1.0, shift_at=20,
                 peak_gap=PEAK_MODEL, peak_new=None, peak_at=20):
        self.h=h0; self.h0=h0; self.lr_mult=lr_mult
        self.phi=phi; self.phi0=phi
        self.lambda_olv=lambda_olv
        self.hysteresis=hysteresis; self.e_crit=e_crit
        self.shift_mult=shift_mult; self.shift_at=shift_at
        self.peak_gap=peak_gap; self.peak_new=peak_new; self.peak_at=peak_at
        self.esfuerzo=0.0

    def tick(self,s):
        if self.shift_mult!=1.0 and s==self.shift_at:
            self.phi=self.phi0*self.shift_mult
        if self.peak_new is not None and s==self.peak_at:
            self.peak_gap=self.peak_new

    @property
    def esf_n(self): return self.esfuerzo/E_REF
    @property
    def h_eff(self): return max(0.5, self.h - self.phi*self.esf_n**1.5)
    @property
    def lr_eff(self):
        # la fatiga degrada la CAPACIDAD de aprender, no solo la dificultad aparente
        return self.lr_mult*max(0.15, 1.0 - 0.7*self.phi*self.esf_n**1.5)

    def expected_gain(self,nivel):
        gap=nivel-self.h_eff; p=rasch_p(self.h_eff,nivel)
        return bell_curve(gap,self.peak_gap)*(p*1.0+(1-p)*0.3)*self.lr_eff
    def in_zpd(self,nivel):
        g=[self.expected_gain(k) for k in NIVELES]; mx=max(g)
        return (self.expected_gain(nivel)>=0.5*mx) if mx>0 else False

    def exercise(self,nivel,rng):
        p=rasch_p(self.h_eff,nivel)
        correcto=rng.random()<p
        gap=max(0.0,nivel-self.h_eff)
        latencia=rng.lognormvariate(math.log(5)+0.6*gap,0.4)
        self.esfuerzo=0.95*self.esfuerzo+latencia/20.0
        gain=bell_curve(nivel-self.h_eff,self.peak_gap)*(1.0 if correcto else 0.3)*self.lr_eff
```

```
        self.h=min(6.0,self.h+gain)
        return {"correcto":correcto,"latencia":round(latencia,2),"gain":gain}

    def descansar(self):
        if self.hysteresis and self.esf_n>self.e_crit: self.esfuerzo*=0.93
        else: self.esfuerzo*=0.7
        self.h=max(0.5,self.h-self.lambda_olv)
```

X.3 Homeostatic agent

The coupling uses the *receiving* drive's pressure, matching Eq. (1); the legacy sender-modulated branch is kept for reference only and used for no reported result.

```
class GCfg:
    def __init__(self,**kw):
        self.lam_p=kw.get("lam_p",0.15); self.lam_c=kw.get("lam_c",0.02)
        self.theta=kw.get("theta",0.7)
        self.alpha_p=kw.get("alpha_p",0.3); self.alpha_c=kw.get("alpha_c",0.3)
        self.w_cp=kw.get("w_cp",-0.05); self.w_pc=kw.get("w_pc",0.05)
        self.g_mode=kw.get("g_mode","learned")
        self.n_drives=kw.get("n_drives",2); self.coupling=kw.get("coupling",True)
        self.coupling_mode=kw.get("coupling_mode",
                "receiver")  # receiver (Eq.1 / design intent) | sender (legacy bug)
        self.eta=kw.get("eta",0.3); self.beta=kw.get("beta",0.2)
        self.g_init_p=kw.get("g_init_p",0.3); self.g_init_c=kw.get("g_init_c",0.5)
        self.g_naive=kw.get("g_naive",0.5)
        self.action_mode=kw.get("action_mode","softmax")   # softmax | argmax
        self.rest_mode=kw.get("rest_mode","gated")  # gated | drive | none
        self.coef=kw.get("coef",(4.0,-3.0,-3.0,1.5,4.0,-3.0))  # up_p,up_c,stay_k,stay_b,down_c,
                down_p          # gated | drive

class GammaAgent:
    def __init__(self,cfg,profile):
        self.cfg=cfg; self.p=profile
        self.d_prog=0.5; self.d_conf=0.5
        self.g_prog={k:cfg.g_init_p for k in NIVELES}
        self.g_conf={k:cfg.g_init_c for k in NIVELES}
        self.acc_ema={}; self.nivel=1; self.frust=0.0
        self.consec_high=0; self.abandono=False; self.dropout_session=None
    def _pressure(self,d): return min(1.0,abs(d-self.cfg.theta)/0.8)
    def basal(self):
        m=self.acc_ema.get(self.nivel,0.5)
        self.d_prog=min(2.0,self.d_prog+self.cfg.lam_p*m)
        if self.cfg.n_drives>=2:
            self.d_conf=min(2.0,self.d_conf+self.cfg.lam_c)
            if self.cfg.coupling:
                pc=self._pressure(self.d_conf); pp=self._pressure(self.d_prog)
                if self.cfg.coupling_mode=="receiver":
                    # Eq.(1): modulated by the RECEIVING drive's own pressure
                    m_to_prog, m_to_conf = pp, pc
                else:
                    m_to_prog, m_to_conf = pc, pp     # legacy: sender's pressure
                if self.d_conf>self.cfg.theta+0.05:
                    self.d_prog=max(0.0,self.d_prog+self.cfg.w_cp*m_to_prog)
                if self.d_prog>self.cfg.theta+0.05:
                    self.d_conf=max(0.0,self.d_conf+self.cfg.w_pc*m_to_conf)
    def select(self,rng,env):
        th=self.cfg.theta
        dp=self.d_prog-th
        dc=(self.d_conf-th) if self.cfg.n_drives>=2 else 0.0
        fn=self.frust/max(self.p["umbral"],0.1)
        strain=max(0.0,fn-0.7)/0.3
        if self.cfg.n_drives>=2:
            if self.cfg.rest_mode=="gated":
                l_rest=2*max(0.0,dc)*strain+4*strain
            elif self.cfg.rest_mode=="none":
                l_rest=-1e9         # rest unavailable: isolates drives from the frustration
                        gate
```

```
            else:
                l_rest=3.0*max(0.0,dc)+2.0*strain-1.2
            c=self.cfg.coef
            logits=[c[0]*dp+c[1]*dc, c[2]*(abs(dp)+abs(dc))+c[3], c[4]*dc+c[5]*dp, l_rest]
        else:
            logits=[4*dp,-3*abs(dp)+1.5,-4*dp,4*strain]
        acts=["subir","mantener","bajar","descansar"]
        if self.cfg.action_mode=="argmax":
            return acts[max(range(4),key=lambda i:logits[i])]
        mx=max(logits); exps=[math.exp(min(50,l-mx)) for l in logits]
        tot=sum(exps); r=rng.random()*tot; cum=0.0
        for i,e in enumerate(exps):
            cum+=e
            if r<cum: return acts[i]
        return "mantener"
    def study(self,env,outcome,nivel):
        c=outcome["correcto"]; eb=self.acc_ema.get(nivel,0.5)
        self.acc_ema[nivel]=(1-self.cfg.beta)*eb+self.cfg.beta*float(c)
        if self.cfg.g_mode=="learned":
            a=self.acc_ema[nivel]
            self.g_prog[nivel]=(1-self.cfg.eta)*self.g_prog[nivel]+self.cfg.eta*(4.0*a*(1.0-a))
            self.g_conf[nivel]=(1-self.cfg.eta)*self.g_conf[nivel]+self.cfg.eta*a
        elif self.cfg.g_mode=="oracle":
            self.g_prog[nivel]=min(1.0,env.expected_gain(nivel)/0.042)
            self.g_conf[nivel]=rasch_p(env.h_eff,nivel)
        if self.cfg.g_mode=="naive": gp=gc=self.cfg.g_naive
        else: gp=self.g_prog[nivel]; gc=self.g_conf[nivel]
        if c:
            self.d_prog=max(0.0,self.d_prog-self.cfg.alpha_p*gp)
            if self.cfg.n_drives>=2:
                self.d_conf=max(0.0,self.d_conf-self.cfg.alpha_c*gc)
            if outcome["latencia"]<15: self.frust=max(0.0,self.frust-0.35)
        else:
            if self.cfg.n_drives>=2: self.d_conf=min(2.0,self.d_conf+0.1)
            self.frust+=0.25
            if eb>=0.7: self.frust+=0.3
```

X.4 Baselines

```
class ReglaAgent:
    def __init__(self,profile,rest=True):
        self.p=profile; self.rest=rest; self.nivel=1; self.frust=0.0
        self.consec_high=0; self.abandono=False; self.dropout_session=None
        self.streak_c=0; self.streak_e=0; self.acc_ema={}
    def select(self,rng,env):
        if self.rest and self.frust>0.6*self.p["umbral"]: return "descansar"
        if self.streak_e>=2: return "bajar"
        if self.streak_c>=3: return "subir"
        return "mantener"
    def study(self,env,outcome,nivel):
        if outcome["correcto"]: self.streak_c+=1; self.streak_e=0
        else: self.streak_e+=1; self.streak_c=0

class BayesianNoObjAgent:
    """State estimation WITHOUT an objective model: tracks h_hat from outcomes and
    simply matches the level to estimated ability. Isolates estimation from
    privileged knowledge of where the learning curve peaks."""
    def __init__(self,profile,rest=True):
        self.p=profile; self.rest=rest; self.nivel=1; self.h_hat=2.5
        self.frust=0.0; self.consec_high=0; self.abandono=False
        self.dropout_session=None; self.acc_ema={}
    def select(self,rng,env):
        if self.rest and self.frust>0.6*self.p["umbral"]: return "descansar"
        best=min(NIVELES,key=lambda k: abs(k-self.h_hat))
        if best>self.nivel: return "subir"
        if best<self.nivel: return "bajar"
        return "mantener"
    def study(self,env,outcome,nivel):
```

```
        p=rasch_p(self.h_hat,nivel)
        self.h_hat+=0.3*(float(outcome["correcto"])-p)
        self.h_hat=max(0.5,min(6.0,self.h_hat))

class BayesianAgent:
    def __init__(self,profile,rest=True):
        self.p=profile; self.rest=rest; self.nivel=1; self.h_hat=2.5
        self.frust=0.0; self.consec_high=0; self.abandono=False
        self.dropout_session=None; self.acc_ema={}
    def select(self,rng,env):
        if self.rest and self.frust>0.6*self.p["umbral"]: return "descansar"
        best,bg=self.nivel,-1
        for k in NIVELES:
            gap=k-self.h_hat; p=rasch_p(self.h_hat,k)
            g=bell_curve(gap)*(p+(1-p)*0.3)
            if g>bg: best,bg=k,g
        if best>self.nivel: return "subir"
        if best<self.nivel: return "bajar"
        return "mantener"
    def study(self,env,outcome,nivel):
        p=rasch_p(self.h_hat,nivel)
        self.h_hat+=0.3*(float(outcome["correcto"])-p)
        self.h_hat=max(0.5,min(6.0,self.h_hat))

class FixedAgent:
    def __init__(self,profile,mode):
        self.p=profile; self.mode=mode; self.nivel=1; self.frust=0.0
        self.consec_high=0; self.abandono=False; self.dropout_session=None
        self.acc_ema={}
    def select(self,rng,env):
        if self.mode=="always1": t=1
        elif self.mode=="always5": t=5
        elif self.mode=="random": t=rng.choice(NIVELES)
        else: t=max(NIVELES,key=lambda k:env.expected_gain(k))
        if t>self.nivel: return "subir"
        if t<self.nivel: return "bajar"
        return "mantener"
    def study(self,env,outcome,nivel): pass

def shared_frust_update(agent,outcome,eb):
    if outcome["correcto"]:
        if outcome["latencia"]<15: agent.frust=max(0.0,agent.frust-0.35)
    else:
        agent.frust+=0.25
        if eb>=0.7: agent.frust+=0.3
```

X.5 Profiles and episode runner

```
def gen_profiles(n=100,seed=2026):
    rng=random.Random(seed)
    return [{"h0":round(rng.uniform(1.5,3.5),2),
             "lr_mult":round(rng.uniform(0.7,1.3),2),
             "umbral":round(rng.uniform(1.8,3.0),1)} for _ in range(n)]

def run_episode(agent,profile,seed,is_gamma,env_kw,log=False):
    env=StudyEnv(profile["h0"],lr_mult=profile["lr_mult"],**env_kw)
    rng=random.Random(seed); traj=[]; zpd=0; nst=0; ndesc=0; s=0
    for s in range(PRESUPUESTO):
        if agent.abandono: break
        env.tick(s)
        if is_gamma: agent.basal()
        a=agent.select(rng,env)
        if a=="subir": agent.nivel=min(5,agent.nivel+1)
        elif a=="bajar": agent.nivel=max(1,agent.nivel-1)
        out=None; inz=False
        if a=="descansar":
            env.descansar(); agent.frust*=0.5; ndesc+=1
        else:
```

```
            inz=env.in_zpd(agent.nivel)
            out=env.exercise(agent.nivel,rng); nst+=1
            if inz: zpd+=1
            if is_gamma: agent.study(env,out,agent.nivel)
            else:
                eb=agent.acc_ema.get(agent.nivel,0.5)
                agent.acc_ema[agent.nivel]=0.8*eb+0.2*float(out["correcto"])
                shared_frust_update(agent,out,eb)
                agent.study(env,out,agent.nivel)
        if agent.frust>=0.95*profile["umbral"]: agent.consec_high+=1
        else: agent.consec_high=0
        if agent.consec_high>=3 and not agent.abandono:
            agent.abandono=True; agent.dropout_session=s
        if log:
            traj.append({"s":s,"nivel":agent.nivel,"action":a,
                "correcto":out["correcto"] if out else None,
                "d_prog":round(getattr(agent,"d_prog",0),3),
                "d_conf":round(getattr(agent,"d_conf",0),3),
                "frust":round(agent.frust,2),"h":round(env.h,3),
                "h_eff":round(env.h_eff,3),"esf_n":round(env.esf_n,3),"in_zpd":inz})
    m={"ganancia":round(env.h-profile["h0"],3),
       "h_final":round(env.h,3),
       "exam":round(rasch_p(env.h,3),3),
       "exam_taken":round(0.0 if agent.abandono else rasch_p(env.h_eff,3),3),
       "exam_eff":round(rasch_p(env.h_eff,3),3),
       "esf_n_final":round(env.esf_n,3),
       "tiz":round(zpd/max(1,nst),3),
       "tib_full":round(zpd/PRESUPUESTO,3),
       "dropout":agent.abandono,"dropout_session":agent.dropout_session,
       "n_descansar":ndesc,"n_study":nst,"nivel_final":agent.nivel,
       "sessions_used":s+1}
    if is_gamma and agent.cfg.g_mode in ("learned","oracle","fixed","naive"):
        gains=[env.expected_gain(k) for k in NIVELES]
        gps=[agent.g_prog[k] for k in NIVELES]
        mg=sum(gains)/5; mp=sum(gps)/5
        cov=sum((gains[i]-mg)*(gps[i]-mp) for i in range(5))
        vg=math.sqrt(sum((g-mg)**2 for g in gains)); vp=math.sqrt(sum((g-mp)**2 for g in gps))
        m["g_validity"]=round(cov/(vg*vp),3) if vg>0 and vp>0 else None
        m["g_prog_final"]={str(k):round(agent.g_prog[k],3) for k in NIVELES}
        m["true_gain_final"]={str(k):round(env.expected_gain(k),4) for k in NIVELES}
    return m,traj

CFG_BASE=dict(lam_p=0.15,alpha_p=0.3,alpha_c=0.3,theta=0.7)
```

Y Analysis code: paired inference and equivalence

The design is within-subject — the same 100 profiles face every arm — so contrasts are paired and equivalence is tested explicitly rather than inferred from non-significance.

```
from scipy.stats import ttest_rel, t as tdist

def per_profile(arm, phi):                  # one unit per profile
    return [mean(run(arm, profile, seed, phi) for seed in SEEDS)
            for profile in PROFILES]

def paired_contrast(a, b, phi):
    xa, xb = per_profile(a, phi), per_profile(b, phi)
    d  = [u - v for u, v in zip(xa, xb)]
    n  = len(d); m = mean(d); se = stdev(d) / sqrt(n)
    h  = se * tdist.ppf(0.975, n - 1)       # 95% CI half-width
    t, p = ttest_rel(xa, xb)                # NOT ttest_ind: paired design
    return m, (m - h, m + h), p, m / stdev(d)

def tost(d, delta):                         # two one-sided tests
```

```
    n = len(d); m = mean(d); se = stdev(d) / sqrt(n)
    return max(1 - tdist.cdf((m + delta) / se, n - 1),
                   tdist.cdf((m - delta) / se, n - 1))

# P3 is an existence claim: max-statistic sign-flip permutation
T_obs = max(paired_t(arm, phi) for arm in GAMMA_ARMS for phi in PHIS_VALID)
null  = [max(paired_t(arm, phi, flip=s) for arm in GAMMA_ARMS
             for phi in PHIS_VALID) for s in random_sign_flips(2000)]
p_P3  = (sum(t >= T_obs for t in null) + 1) / 2001
```

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and Introduction state the split answer (g is learnable; the learned signal does not reach behavior), the viability result, and the RLP decomposition, matching the Results section exactly. The abstract reports the three-way decomposition with effect sizes, states that a learned signal underperforms a constant, reports the refuted prediction P7, and states that the crossover result concerns robustness, and the Limitations section bounds the claim by noting the crossover occurs in a low-gain regime.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: The dedicated Limitations section covers the low-gain regime in which the crossover occurs, incomplete operationalization of signal validity (only g_{prog}, terminal, five points), the functional-form mismatch that caps achievable validity and partially confounds the oracle contrast, the fact that policy tuning was a random search rather than an optimum (making the residual an upper bound), the still-untested applicability conditions (2) and (4), the self-attested commitment hash and the un-committed gate thresholds, and simulator simplicity.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.

- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: The paper presents no theorems or formal proofs. Its contributions are empirical, and the equations given in the Architecture section and Appendix B are model definitions rather than theoretical results requiring proof.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Appendix A gives the complete environment specification with all constants; Appendix B gives the agent configuration and the action-selection code; Appendix F documents the seed derivation procedure (SHA-256 from master seed 20260820), the profile generation, and the compute requirements. The Experimental Design section specifies the arm definitions and the paired statistical protocol. Sample sizes differ by experiment and are stated separately: the main sweep uses 100 profiles $\times$ 10 seeds $\times$ 10 arms $\times$ 6 adversity points (60,000 episodes); coefficient tuning scores on a 20-profile subset and validates on 50 strictly held-out profiles; the post-hoc maintenance experiment uses 60 profiles $\times$ 6 seeds. An anonymized reproducible notebook is included in the supplementary material, and the SHA-256 commitment hash of the preregistered predictions is reported in the paper and recomputable from the verbatim prediction text in the appendix.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case

of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.

- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: An anonymized self-contained notebook reproducing every reported number, table, and figure is included in the supplementary material. No external dataset is required: the environment is a simulator fully specified in Appendix A, and all stochasticity is controlled by the documented seed derivation. Dependencies are limited to `scipy` and `matplotlib`.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The Experimental Design section specifies the adversity axis and its sweep values, the ten arms, the episode protocol (100 profiles $\times$ 10 seeds $\times$ 10 arms $\times$ 6 points), and the profile-clustered statistical procedure. Appendix B lists every agent hyperparameter. The same section documents the quality gate applied at each sweep point and states that results are reported only over the passing range.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: The design is within-subject (the same 100 profiles face every arm), so all inferential comparisons use paired t-tests on profile-level means (n=100), reporting the paired difference, its 95% confidence interval, and the paired effect size d_z. Claims resting on the absence of an effect are additionally supported by TOST equivalence tests against a prespecified smallest effect size of interest (δ=0.05), rather than inferred from non-significance. Confidence intervals are reported for every headline contrast, including the crossover. The clustering unit and test are stated in the Experimental Design section.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: A dedicated Compute Resources section states the machine (Intel Core i7-1165G7, 4 cores, 32 GB RAM, no GPU), the wall-clock time for the main sweep (60,000 episodes in 31.4 s single-threaded), the coefficient search (3 min), and the per-decision cost (14 μs), together with the operator's $O(n^2 + A)$ per-session arithmetic and its lack of gradients, value functions, or replay. Exploratory and discarded runs during development were of the same negligible per-run cost, and the entire project consumed less compute than a single GPU-minute.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The research uses only a synthetic simulator with no human subjects, no personal data, and no scraped assets. Anonymity is preserved throughout the submission. Potential dual-use concerns arising from modeling learner internal states are discussed in the Impact Statement.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: The Impact Statement section discusses the positive case (tutoring systems that respect cognitive and emotional limits, supported directly by the paper's viability results) and the negative case (the same mechanism inverted into an engagement-maximizing system that learns when to intervene to keep a user hooked), and identifies transparency about whose homeostasis is regulated as the mitigation.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.

- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The released artifact is a small simulation notebook implementing a synthetic environment and a two-drive regulator. It contains no pretrained model, no generative capability, and no data, and therefore poses no misuse risk requiring safeguards.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [N/A]

    Justification: No existing datasets, code packages, or models are used beyond standard open-source scientific libraries (`scipy`, BSD-3-Clause; `matplotlib`, PSF-based license), used in their ordinary capacity. The conceptual sources underlying the environment, notably the Rasch model and the Zone of Proximal Development, are cited in the references.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

Justification: The new asset is the simulation notebook, provided anonymized in the supplementary material. It is documented inline with markdown cells covering the research question, the preregistered predictions, the rationale for each design change, the quality gate criteria, and the prediction verdicts, and it emits a structured JSON of all results.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper involves no crowdsourcing and no human subjects. The "student" is a simulated agent in a synthetic environment; no data from real learners is collected or used.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: No human subjects research was conducted, so no IRB approval or equivalent review was required.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: We disclose LLM usage for transparency although it does not constitute a core-method component. No language model participates in the agent, the environment, the baselines, or the analysis: all are explicit numerical simulations. LLM assistance was confined to (i) code assistance, using Devin as an IDE with Kimi 3 and DeepSeek v4 Pro, and Claude Opus 5 and Claude Fable 5, and (ii) writing and editing assistance, using Claude Opus 5 and Claude Fable 5. All experimental design decisions, preregistered predictions, statistical procedures, and interpretations are the authors' own, and every reported number was produced by executing the released notebook.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.